Servidor HTTP/1.1 com QoS em C

Explicação completa do projeto (V1 e V2)

Giovana Piazza Luis Felipe Maccalli
Vitor Moreira

Fundamentos e Avaliação de Redes de Computadores — 2026/2 — Unipampa

Outubro de 2026

Resumo

Este documento explica, de forma didática, todo o projeto desenvolvido no trabalho de programação em rede: um servidor HTTP/1.1 escrito em C, com sockets TCP e Pthreads, que atende vários clientes ao mesmo tempo (versão 1) e aplica mecanismos de Qualidade de Serviço — taxa por IP, divisão de banda, controle de admissão e painel em tempo real (versão 2). São descritos a organização dos arquivos, a arquitetura, o funcionamento de cada parte do código, como compilar, executar e testar, e as limitações conhecidas.

Sumário

1 Visão geral **3**

2 Organização dos arquivos **3**

3 Conceitos de rede usados **4**

4 Arquitetura **4**

5 Versão 1 — servidor HTTP/1.1 concorrente **5**
5.1 Leitura do cabeçalho (`read_header`) . . . 5
5.2 Interpretação da requisição (`parse_request`) . . . 5
5.3 Localização do arquivo (`open_resource`) . . . 5
5.4 Envio da resposta . . . 6
5.5 Logs e encerramento . . . 6
5.6 Limites técnicos . . . 6

6 Versão 2 — Qualidade de Serviço **6**
6.1 Arquivo de taxas . . . 6
6.2 Tabela de IPs (`InfoIP`) . . . 6
6.3 Controle de admissão . . . 7
6.4 Envio com taxa controlada . . . 7
6.5 Divisão da taxa entre conexões do mesmo IP . . . 7
6.6 Estimativa de RTT, atraso e banda . . . 7
6.7 Painel em tempo real . . . 8
6.8 Sincronização . . . 8

7 Como compilar e executar **8**

8 Testes **9**
8.1 V1 — suíte de integração 9
8.2 V2 — `teste.sh` 9
8.3 Experimentos com ferramentas de rede 9

9 Limitações e problemas conhecidos **9**

Referências **10**

1 Visão geral

O enunciado da disciplina pede um servidor HTTP para Linux, em linguagem C com Pthreads, desenvolvido em versões incrementais (MVPs):

V1 — servidor concorrente: atender vários clientes simultaneamente seguindo o HTTP/1.1, com *conexões persistentes* (várias requisições no mesmo socket TCP).

V2 — Qualidade de Serviço (QoS): enviar os objetos (exceto o HTML) com taxa controlada conforme o IP do cliente; dividir a taxa entre conexões do mesmo IP; aplicar controle de admissão com base na vazão máxima do servidor; e mostrar em tempo real os clientes atendidos, com estimativas de atraso fim-a-fim e largura de banda baseadas no RTT.

O servidor entrega somente **conteúdo estático** (arquivos de uma pasta), o que basta para observar o comportamento das camadas de aplicação (HTTP) e transporte (TCP). A página de teste contém duas imagens BMP de aproximadamente 12 MiB cada, para que as transferências durem tempo suficiente para serem medidas e controladas.

Tabela 1: Resumo do que cada versão implementa.

Versão	Funcionalidades
V1	Socket TCP IPv4, uma thread por conexão, GET e HEAD, `index.html` padrão, Content-Length e tipos MIME, conexão persistente, `Connection: close`, pipelining, respostas de erro, proteção de caminhos, timeouts, encerramento por sinal.
V2	Tudo da V1 + arquivo de taxas por IP, taxa padrão de 1000 kbps, envio com taxa controlada para não-HTML, divisão da taxa entre conexões do mesmo IP, controle de admissão por vazão máxima, painel no terminal com RTT, atraso e banda estimados.

2 Organização dos arquivos

```
Redes_De_Comp/
|-- README.md # readme da V1
|-- V1/ # primeira versao (MVP1)
| |-- Makefile
| |-- src/server.c # servidor concorrente HTTP/1.1
| |-- www/index.html # pagina com duas imagens
| |-- tools/generate_images.py # gera imagem1.bmp e imagem2.bmp (~12 MiB)
| |-- tools/inspect_pcap.py # le capturas PCAP
| `-- tests/test_server.py # 26 testes de integracao
|-- V2/ # segunda versao (MVP2, QoS)
| |-- Makefile
| |-- README.md
| |-- src/server.c # V1 + QoS
| |-- taxas.txt # IP e taxa em kbps
| |-- teste.sh # teste simples com curl
| |-- www/index.html
| `-- tools/generate_images.py
|-- evidencias/ # resultados dos testes da V1
`-- documentos/ # plano de experimentos e este documento
```

As imagens BMP não ficam no Git (são grandes); elas são geradas com `make fixtures`.

3 Conceitos de rede usados

Sockets TCP. O servidor usa a API de sockets do Linux. A sequência é `socket()` → `bind()` (associa à porta) → `listen()` → `accept()` (uma vez por cliente). Cada `accept()` devolve um novo descritor, usado com `recv()` e `send()`. O TCP garante entrega ordenada e confiável de um fluxo de bytes; ele *não* preserva fronteiras de mensagem, por isso o servidor precisa procurar o fim do cabeçalho HTTP por conta própria.

HTTP/1.1. Uma requisição é composta por uma linha inicial (`GET /imagem1.bmp HTTP/1.1`), campos de cabeçalho (`Host`, `Connection`...) e uma linha vazia (`\r\n\r\n`). A resposta tem a linha de status (`HTTP/1.1 200 OK`), cabeçalhos e o corpo.

Conexões persistentes. No HTTP/1.1 a conexão fica aberta por padrão (*keep-alive*), e o cliente pode enviar várias requisições no mesmo socket. Para o cliente saber onde termina cada resposta, o servidor sempre manda `Content-Length`. O cliente encerra enviando `Connection: close`.

Threads (Pthreads). Cada conexão é atendida por uma thread própria (`pthread_create`). Assim, um cliente lento não bloqueia os demais. Dados compartilhados entre threads (tabelas, contadores, logs) são protegidos por um *mutex* (`pthread_mutex_t`).

QoS e controle de taxa. Qualidade de Serviço aqui significa limitar quanto de banda cada cliente pode usar e não aceitar mais clientes do que o servidor consegue atender. Taxas são expressas em kbps (1 kbps = 1000 bits por segundo).

4 Arquitetura

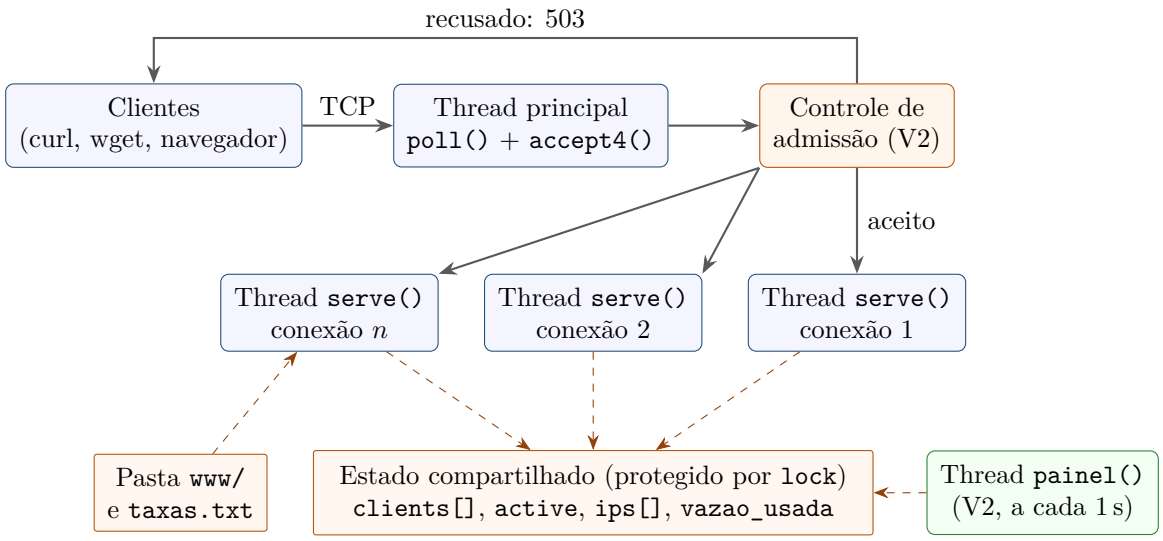


Figura 1: Visão geral das threads e dos dados compartilhados do servidor.

1. A **thread principal** cria o socket de escuta e fica em um laço com `poll()` (espera de 500 ms, para conseguir perceber o pedido de parada) e `accept4()`.
2. Para cada novo cliente, procura um espaço livre em `clients[]` (máximo de 128 conexões). Na V2, também passa pelo **controle de admissão**.
3. Se aceito, aloca uma estrutura `Client` (descritor, posição na tabela, IP) e cria uma thread que executa `serve()`. A thread é *detached*: libera os próprios recursos ao terminar.

4. Cada thread `serve()` executa o ciclo *ler cabeçalho* → *interpretar* → *responder* → *ler de novo* no mesmo socket, até o cliente fechar, pedir `close`, ocorrer erro ou atingir 100 requisições.
5. Na V2, a thread `painel()` redesenha a tela a cada segundo.

5 Versão 1 — servidor HTTP/1.1 concorrente

5.1 Leitura do cabeçalho (`read_header`)

Como o TCP é um fluxo de bytes, uma requisição pode chegar em vários pedaços ou várias requisições podem chegar juntas. A função mantém um buffer de até 16 KiB por conexão e:

- procura a sequência `\r\n\r\n`; quando encontra, devolve o tamanho do cabeçalho;
- se o buffer enche sem achar o fim, responde **431** (cabeçalho grande demais);
- se passarem 10 s sem completar o cabeçalho, responde **408** (timeout);
- os bytes que sobram depois do cabeçalho são mantidos no buffer (com `memmove`), pois pertencem à próxima requisição. É isso que permite o *pipelining*.

5.2 Interpretação da requisição (`parse_request`)

Separa método, alvo e versão da primeira linha e percorre os campos de cabeçalho. As principais regras são:

Situação	**Resposta**
Versão diferente de `HTTP/1.1`	505 HTTP Version Not Supported
Método diferente de GET/HEAD	405 Method Not Allowed (com `Allow: GET, HEAD`)
`Host` ausente, duplicado ou inválido	400 Bad Request
Alvo maior que 4096 bytes	414 URI Too Long
`Transfer-Encoding`	501 Not Implemented
`Expect`	417 Expectation Failed
Corpo (`Content-Length` $\neq 0$) ou framing ambíguo	400 e fecha a conexão
`Connection: close`	marca para fechar após a resposta

Recusar corpos é uma decisão de segurança: como o servidor não lê corpo, se aceitasse um, os bytes do corpo seriam interpretados como a próxima requisição.

5.3 Localização do arquivo (`open_resource`)

- a *query string* (depois de `?`) é ignorada;
- sequências `%XX` são decodificadas; inválidas geram 400;
- se o caminho termina em `/`, usa `index.html`;
- o caminho é aberto componente por componente com `openat()` e `O_NOFOLLOW`, a partir do descritor da pasta raiz. Componentes `..` ou `.` geram 403. Isso impede sair da pasta `www/` (*path traversal*) e seguir links simbólicos;
- apenas arquivos regulares são enviados (diretórios geram 403); arquivo inexistente gera 404.

5.4 Envio da resposta

A função `header()` monta a linha de status e os campos `Date`, `Server`, `Content-Type` (pela extensão: html, css, js, png, jpg, bmp...), `Content-Length` e `Connection`. Em GET, o arquivo é lido e enviado em blocos de até 64 KiB; em HEAD, somente os cabeçalhos. A função `send_all()` repete o `send()` até enviar tudo, porque o kernel pode aceitar só parte dos bytes (*escrita parcial*). O sinal `SIGPIPE` é ignorado para que um cliente que fecha a conexão no meio não derrube o processo.

5.5 Logs e encerramento

Cada requisição gera uma linha de log com IP, socket, número da requisição, método, alvo, status, bytes e conexões ativas. `SIGINT` (Ctrl+C) e `SIGTERM` ativam a variável `stopping`; o laço principal termina, os sockets dos clientes recebem `shutdown()` e a thread principal espera (variável de condição `empty`) até todas as threads saírem.

5.6 Limites técnicos

128 conexões simultâneas (acima disso: 503), 100 requisições por conexão, cabeçalho de 16 KiB, 10 s para completar cada cabeçalho e timeout de envio (`SO_SNDTIMEO`). Esse limite de 128 protege os recursos do processo; ele **não** é o controle de admissão por vazão da V2.

6 Versão 2 — Qualidade de Serviço

As alterações da V2 estão todas em `V2/src/server.c` e marcadas com comentários.

6.1 Arquivo de taxas

O arquivo `taxas.txt` tem uma linha por cliente, no formato `IP KBPS`:

```
127.0.0.1 2000
127.0.0.2 3000
192.168.0.10 500
```

A função `carrega_taxas()` lê o arquivo na inicialização com `fscanf` (até 100 linhas) e `busca_taxa()` procura o IP do cliente. Se o IP não estiver cadastrado, usa a taxa padrão de **1000 kbps**, como pede o enunciado. Se o arquivo não existir, todos os clientes recebem 1000 kbps.

6.2 Tabela de IPs (`InfoIP`)

Para dividir a banda e alimentar o painel, o servidor mantém uma tabela `ips[]` com uma entrada por IP:

Campo	Significado
`conexoes`	número de conexões abertas desse IP agora
`taxa`	taxa total do IP (kbps), dividida entre as conexões
`t_html`, `tam_html`	momento (ms) e tamanho do último HTML pedido
`rtt`, `banda`	estimativas de RTT (ms) e de banda do cliente (kbps)
`taxa_real`	taxa medida no envio em andamento (kbps)
`bytes`	total de bytes de objetos enviados para o IP

A entrada não é apagada quando o cliente desconecta, para não perder o tempo do HTML caso o cliente (por exemplo, o `wget`) abra outra conexão para baixar as imagens. Uma entrada com zero conexões pode ser reaproveitada por outro IP (função `acha_ip()`).

6.3 Controle de admissão

A vazão máxima do servidor é o terceiro parâmetro da linha de comando. A variável `vazao_usada` guarda a soma das taxas dos IPs que estão sendo atendidos. Quando uma conexão chega:

1. se o IP **já tem conexão aberta**, ele é aceito sem somar nada, pois a nova conexão vai apenas dividir a taxa que o IP já tem;
2. se é um **IP novo**, ele só é aceito se

$$\texttt{vazao_usada} + \texttt{taxa_do_ip} \leq \texttt{vazao_max};$$

   nesse caso `vazao_usada` aumenta pela taxa do IP;
3. caso contrário, o servidor responde **503 Service Unavailable** e fecha a conexão.

Quando a última conexão de um IP termina, a taxa dele é devolvida (`vazao_usada -= taxa`). A interpretação de "1000 kbps se viável" adotada pela equipe foi: se não couber, o cliente é recusado.

6.4 Envio com taxa controlada

O HTML é enviado sem limite (como pede o enunciado). Para os demais objetos, o arquivo é enviado em blocos de 8 KiB e, depois de cada bloco, a thread dorme o tempo que esse bloco levaria na taxa permitida. Para um bloco de n bytes e taxa R em kbps:

$$t_{\text{espera}}\ [\mu s] \;=\; \frac{8n \text{ bits}}{R \times 1000 \text{ bits/s}} \times 10^6 \;=\; \frac{8n \times 1000}{R}.$$

Exemplo: $n = 8192$ e $R = 1000$ kbps resultam em $65\,536\,\mu s \approx 65{,}5$ ms por bloco, ou seja, cerca de 125 KB/s.

```
long taxa_con = ips[c->ip_idx].taxa / ips[c->ip_idx].conexoes;
long espera = (long)n * 8 * 1000 / taxa_con; /* em microssegundos */
usleep((useconds_t)espera);
```

6.5 Divisão da taxa entre conexões do mesmo IP

A taxa de cada conexão é $R_{\text{con}} = R_{\text{IP}} / N_{\text{conexões}}$. Esse valor é recalculado **a cada bloco**, então se o cliente abre uma segunda conexão no meio de um download, as duas passam a usar metade da taxa a partir do bloco seguinte. Se uma delas termina, a outra volta à taxa cheia.

6.6 Estimativa de RTT, atraso e banda

Seguindo o enunciado, o RTT é estimado pelo intervalo entre duas requisições consecutivas do mesmo cliente: a do HTML e a do primeiro objeto referenciado nele.

- quando chega o pedido do HTML, guarda-se o instante t_{html} e o tamanho do HTML;
- quando chega o primeiro pedido de um objeto não-HTML desse IP: RTT $\approx t_{\text{objeto}} - t_{\text{html}}$ (mínimo de 1 ms, para evitar divisão por zero no *localhost*);
- **atraso fim-a-fim** $\approx$ RTT/2;
- **banda estimada** $\approx \dfrac{8 \times \text{tamanho do HTML (bytes)}}{\text{RTT (ms)}}$ em kbps (bits por milissegundo equivalem a kbps).

Limitação da estimativa

O intervalo medido não é um RTT puro: ele inclui o tempo de transmissão do HTML, o tempo de o cliente interpretar a página e decidir pedir a imagem. Por isso é uma *estimativa*, não uma medição direta do atraso unidirecional. Navegadores podem nem pedir as imagens ou pedi-las em outra ordem (por exemplo, `favicon.ico`); por isso o enunciado recomenda `curl` e `wget`.

6.7 Painel em tempo real

A thread `painel()` limpa o terminal com o código ANSI `\033[2J\033[H` e, a cada segundo, mostra a vazão usada e uma linha por IP com conexões abertas. Exemplo real, com duas conexões de 127.0.0.1 dividindo 2000 kbps:

```
===== Servidor HTTP com QoS (V2) =====
vazao usada: 2000 / 4000 kbps   conexoes ativas: 2

IP          CON  TAXA_IP  TAXA/CON  RTT(ms)  ATRASO(ms)  BANDA(kbps)  REAL(kbps)  ENVIADO
127.0.0.1     2     2000      1000      1.0         0.5       4256.0      1023.6    608KB
```

Como a tela é do painel (`stdout`), os logs de requisição passaram a ir para o `stderr`; para guardá-los use `2> server.log`. Conexões recusadas aparecem no log como `RECUSADO ip=...`.

6.8 Sincronização

Todos os acessos a `clients[]`, `active`, `ips[]` e `vazao_usada` são feitos com o mutex `lock`. A admissão (escolher o espaço, verificar e somar a taxa, incrementar as conexões do IP) é feita inteira dentro de uma única seção crítica, para que dois clientes chegando juntos não ultrapassem a vazão máxima. O `usleep` é chamado *fora* do lock, senão uma conexão dormindo travaria todas.

7 Como compilar e executar

Dependências: Linux, GCC, Make, Python 3 (para gerar as imagens) e `curl`.

```
sudo apt install build-essential python3 curl tcpdump wireshark iptraf-ng
```

V1

```
cd V1
make && make fixtures
./server 8080 www # PORTA DIRETORIO_WEB
```

V2

```
cd V2
make && make fixtures
./server 8080 www 5000 taxas.txt 2> server.log
# PORTA DIR VAZAO_MAX_KBPS [ARQUIVO_TAXAS]
```

Para ver o RTT no painel, peça o HTML e uma imagem na mesma execução do curl:

```
curl -o /dev/null http://localhost:8080/ -o /dev/null http://localhost:8080/imagem1.bmp
```

De outro computador, use `http://IP_DO_SERVIDOR:8080/` e libere a porta no firewall. Uma imagem de 12 MiB a 1000 kbps leva cerca de 100 s.

8 Testes

8.1 V1 — suíte de integração

`make test` executa `tests/test_server.py`, que inicia o servidor real em uma porta livre e conversa com ele por sockets TCP. Foram aprovados **26 de 26 testes**, incluindo: quatro requisições no mesmo socket, pipelining (200, 404, 200 em ordem), cabeçalho fragmentado, integridade das imagens por SHA-256, HEAD, 12 clientes concorrentes, métodos inválidos, `Host` ausente/duplicado, versão não suportada, path traversal, link simbólico, cabeçalho grande, limite de requisições e timeout de 10 s. Os resultados estão em `evidencias/`.

8.2 V2 — `teste.sh`

`make teste` executa um script simples que sobe o servidor na porta 8090 com vazão máxima de 4000 kbps, cria um arquivo `www/teste.bin` de 500 KB e faz quatro testes:

Teste	Esperado	Obtido
1. Objeto a 2000 kbps	≈ 2 s e 250 000 B/s	2,04 s; 250 568 B/s
2. Duas conexões do mesmo IP	≈ 4 s cada (1000 kbps cada)	4,11 s e 4,11 s
3. Admissão: 127.0.0.2 (3000) com 2000 em uso	503	503
4. HTML sem limite	muito rápido	0,0002 s

O teste 3 usa `curl -interface 127.0.0.2`: no Linux toda a faixa `127.0.0.0/8` é *loopback*, o que permite simular um segundo IP na mesma máquina.

8.3 Experimentos com ferramentas de rede

O arquivo `documentos/plano-experimentos.md` descreve o procedimento planejado: capturar com `tcpdump -i INTERFACE tcp port 8080 -w experimento.pcap`, analisar no Wireshark (filtro `tcp.port == 8080`, *Follow TCP Stream*, *Decode As HTTP*), acompanhar a vazão no `iptraf-ng` e repetir com 1, 4 e 12 clientes, em rede cabeada e Wi-Fi. Na V2, o esperado é ver no IPTraf a vazão de cada cliente próxima da taxa do `taxas.txt`. Essas medições ainda precisam ser feitas pela equipe.

9 Limitações e problemas conhecidos

- Servidor didático: só conteúdo estático, só IPv4, sem HTTPS, sem corpo de requisição e sem *chunked*.
- A taxa real fica um pouco abaixo da configurada (ex.: cerca de 1023 kbps medidos contra 1000 configurados, e um pouco menos em redes reais), porque o `usleep` não desconta o tempo gasto no `send`.
- A divisão da taxa é inteira (1000/3 = 333 kbps).
- O arquivo de taxas não aceita comentários nem linhas fora do formato: a leitura para na primeira linha inválida.
- O valor da vazão máxima é lido com `atol`, sem validação completa.
- O painel segura o mutex enquanto imprime, o que pode atrasar um pouco as outras threads.
- O RTT é uma estimativa (ver seção de RTT); a banda estimada depende do tamanho do HTML.
- O *keep-alive* fecha após 10 s sem nova requisição.

- No `teste.sh`, a primeira linha do teste 1 aparece como "objeto:" em vez de "html:", pois com várias URLs o curl usa apenas o último `-w`.

Uso de IA

Conforme exigido pela especificação, partes do código da V2 foram produzidas com apoio de IA (Claude) e revisadas pela equipe; isso está indicado no topo de `V2/src/server.c` e no `V2/README.md`. Todos os integrantes devem entender o código antes da demonstração.

Referências

- Fundamentos e Avaliação de Redes de Computadores (2026). *Trabalho de programação em rede: especificação do trabalho prático*. 2026/2.
- Forouzan, B. A.; Mosharraf, F. (2013). *Redes de computadores: uma abordagem top-down*. Porto Alegre: AMGH.
- Fielding, R.; Nottingham, M.; Reschke, J. (2022). *HTTP/1.1*. RFC 9112. IETF. `https://www.rfc-editor.org/rfc/rfc9112`
- Fielding, R.; Nottingham, M.; Reschke, J. (2022). *HTTP Semantics*. RFC 9110. IETF. `https://www.rfc-editor.org/rfc/rfc9110`
- Lawrence Livermore National Laboratory. *POSIX Threads Programming*. `https://computing.llnl.gov/tutorials/pthreads/`
- Repositório da equipe: `https://github.com/giovanapiazza/Redes_De_Comp`